



AI Agents & Agentic AI

Autonomous AI Systems

IN THIS WEEK · 3 TOPICS · 37 SECTIONS

01

AI Agent Fundamentals

13 SECTIONS

- 01 What is an AI Agent
- 02 AI Agent Architecture
- 03 Agent Loop
- 04 Perception
- 05 Reasoning
- 06 Planning
- 07 Acting
- 08 Reflection
- 09 Memory
- 10 Environment
- 11 Agent Lifecycle
- 12 Single Agent Systems
- 13 Multi-Agent Systems

02

Agent Frameworks

13 SECTIONS

- 01 LangChain
- 02 LangGraph
- 03 AutoGen
- 04 CrewAI
- 05 OpenAI Agents SDK
- 06 Google ADK
- 07 Semantic Kernel
- 08 PydanticAI
- 09 SmolAgents
- 10 Agno
- 11 DSPy
- 12 Mastra
- 13 Atomic Agents

03

Agent Patterns

11 SECTIONS

- 01 ReAct Pattern
- 02 Planner Pattern
- 03 Executor Pattern
- 04 Router Pattern
- 05 Supervisor Pattern
- 06 Reflection Pattern
- 07 Debate Pattern
- 08 Human-in-the-Loop
- 09 Swarm Agents
- 10 Hierarchical Agents
- 11 Event-Driven Agents

WEEK 08 OF 17

01

02

03

04

05

06

07

08

09

10

11

12

13

14

15

16

17

AI Agent Fundamentals

01 What is an AI Agent

At its core, an AI agent is an AI system that can autonomously take actions in order to accomplish a goal, rather than just generating a single response to a single question. The word "autonomously" is doing a lot of work here — it means the system itself is making decisions about what to do next, rather than a human explicitly telling it every single step along the way.

To make this concrete, let's compare a regular AI chatbot interaction with an agent. If you ask a regular chatbot "what's the weather going to be like tomorrow," it can only answer based on whatever knowledge it already has baked in from training — and since weather is constantly changing and specific to the moment, a basic chatbot genuinely can't answer this well at all. An AI agent, on the other hand, given the same question, might realize on its own that it doesn't know the current weather, decide to use a weather-lookup tool to actually check a live weather service, read the result that comes back, and then use that real, current information to give you an accurate answer.

The key ingredients that make something an "agent" rather than just a chatbot are: the ability to take actions (not just generate text, but actually do things — search, calculate, call other software), the ability to use tools (external capabilities beyond its own built-in knowledge, like a calculator, a search engine, a code execution environment, or a company's internal database), and some degree of independent decision-making about what steps are needed and in what order, rather than following a single, fixed, pre-written script every time.

It's worth being clear that "agent" isn't a single precise technical term with one universally agreed definition — you'll see it used somewhat loosely across the industry, sometimes referring to fairly simple systems that can just use one or two tools, and sometimes referring to much more sophisticated systems that can plan multi-step projects, remember things across long periods of time, and coordinate with other agents. But the common thread running through all of it is this idea of an AI system that can take real actions toward a goal, rather than just answering questions.

02 AI Agent Architecture

Now that we understand what an agent is at a conceptual level, let's look at what's actually happening inside one — the different pieces that need to work together to make this autonomous, goal-pursuing behavior possible.

At the center of almost every AI agent is a language model (like GPT or Claude) acting as the "brain" — the component responsible for understanding the goal, deciding what to do, and interpreting results. But the language model alone isn't enough to make something a full agent; it needs several supporting pieces built around it.

There's typically a **tools layer** — a defined set of specific actions the agent is actually allowed to take, like "search the web," "read a file," "run this piece of code," or "send an email." Each tool usually has a clear description of what it does and what information it needs, so the language model can understand when and how to use it appropriately.

There's a **memory component** — a way for the agent to keep track of what's happened so far during its current task (and sometimes across multiple separate tasks over time), so it doesn't lose track of what it's already tried, what it's already learned, or what the original goal even was, especially during a long, multi-step task. We'll dig into memory more deeply a bit later in this explanation.

There's an **orchestration or control layer** — the logic that actually manages the overall flow, deciding when to call the language model, when to execute a tool, when to feed a tool's result back to the language model, and when the task is finally considered complete. This is essentially the "wiring" that connects all the other pieces together into a working system.

And there's often an **environment** — the actual context the agent is operating within, whether that's a web browser it can control, a file system it can read and write to, a company's software systems it can interact with, or a sandboxed coding environment. We'll also come back to this concept later.

Together, these pieces form the architecture that allows an agent to do something a plain language model alone cannot: perceive information beyond its training data, make a decision about what to do, actually do it, observe what happened, and use that observation to inform its next decision — repeating this cycle until the goal is achieved.

03 Agent Loop

The agent loop is the repeating cycle of steps an agent goes through as it works on a task, and it's really the heartbeat of how an agent operates. Understanding this loop is one of the most useful things for building a solid mental model of how agents actually function, because nearly everything else we discuss in this guide fits into one part of this loop.

At a high level, the loop generally looks like this: the agent **observes** its current situation (what's the goal, what's happened so far, what information is available right now), it **decides** what action to take next based on that observation, it **executes** that action (like calling a tool), it **observes the result** of that action, and then the whole cycle repeats — using this new information to decide the next action — continuing on and on until the agent determines the goal has actually been achieved, or until some limit is reached (like a maximum number of steps, to prevent the agent from looping forever).

A simple, concrete way to picture this: imagine you ask an agent to "find out how many days until my flight and tell me if I need to pack a jacket based on the weather at my destination." The loop might go: observe the goal → decide it first needs to find the flight date → take the action of checking a calendar tool → observe the result (the flight is in 12 days) → decide it now needs the destination weather → take the action of calling a weather tool → observe the result (cold weather expected) → decide it now has enough information → take the final action of generating a summary answer for the user → done.

This looping structure is exactly what allows an agent to handle tasks that can't be solved in a single step. A basic chatbot only gets one shot to generate a response based on the original question. An agent, through this repeating loop, can gather information, act, observe, and adjust — as many times as genuinely needed — before arriving at a final answer or completing a task.

04 Perception

Perception refers to how an agent takes in and understands information about its current situation — essentially, how it "senses" the world it's operating in, similar in spirit to how the word is used when describing human senses like sight and hearing, just applied to an AI system's much more limited and

specific ways of gathering information.

For an AI agent, perception typically comes through a few channels. There's the **original instruction or goal** given by the user — this is the starting point of what the agent needs to understand and work toward. There's **information returned from tools** — when an agent calls a tool (like a web search or reading a file), the result that comes back is a form of perception, giving the agent new information about the world beyond what it already knew. There's also often the **current state of the environment** it's working in — for example, an agent working within a code repository perceives the current contents of the files it's looking at, or an agent controlling a web browser perceives the current contents of the webpage it's looking at.

It's worth understanding that an AI agent's perception is fundamentally limited compared to a human's — it can only "see" or "know" whatever specific information gets explicitly fed into it, whether that's the original text of the goal, the specific output of a tool it called, or specific content it was given access to. Unlike a human who's continuously and passively absorbing all sorts of ambient information through their senses all the time, an AI agent's perception is much more deliberate and narrow — it only perceives exactly what's been explicitly provided to it or what it's explicitly retrieved through a specific action. This is actually an important thing to keep in mind when designing agent systems well — if an agent seems to be making a poor decision, it's often because it simply didn't have access to some piece of information a human would have naturally picked up on, rather than the agent being fundamentally unable to reason about that information if it had actually been given it.

05 Reasoning

Reasoning refers to the agent's process of thinking through a problem — working through the logic of a situation, weighing different options, and drawing conclusions — before deciding what to actually do. This is the part of the agent loop that happens "inside the agent's head," so to speak, connecting what it has perceived to what it ultimately decides to do next.

For AI agents built on language models, reasoning generally happens through the model generating text that works through a problem step by step, rather than jumping immediately to a final answer or action. You may have heard of the term "chain of thought," which refers to exactly this — encouraging the AI model to write out its reasoning process explicitly, step by step, before arriving at a conclusion or a decision about what action to take, since models tend to produce much better, more accurate final decisions when they work through the logic first, rather than trying to jump straight to an answer.

Good reasoning in an agent context often involves things like: breaking a complex situation down into smaller, more understandable pieces, weighing multiple possible next actions against each other and considering which one is actually most likely to move the task forward, noticing when something unexpected or contradictory has happened and figuring out why, and generally connecting the dots between the original goal, everything perceived so far, and what logically needs to happen next.

It's worth understanding that reasoning and planning (the next topic we'll cover) are closely related and sometimes talked about somewhat interchangeably, but there's a useful distinction: reasoning is more about the moment-to-moment logical thinking involved in interpreting a situation and deciding on an immediate next step, while planning tends to refer to something a bit more forward-looking and structured — mapping out a fuller sequence of intended steps in advance. Let's look at planning next.

06 Planning

Planning refers to an agent's ability to think ahead and map out a sequence of steps needed to accomplish a goal, rather than only ever deciding on the single very next immediate action with no broader sense of the overall path forward.

Why does this distinction matter? Some simpler agent designs are very "reactive" — at each point in the loop, they only figure out the single next step, without any real plan for the steps beyond that, and they may end up circling back and forth inefficiently, or missing an important step until they stumble onto the need for it later. More sophisticated agents instead try to form an actual plan early on — something like "to accomplish this goal, I'll need to do step A, then step B, then step C" — mapped out before diving into execution, similar to how a person planning a road trip might sketch out the overall route and the cities they'll pass through, before actually starting to drive, rather than just picking a direction and figuring out each turn only once they arrive at it.

Good planning tends to make an agent noticeably more effective at handling complex, multi-step tasks, because it helps the agent stay organized, avoid unnecessary repeated or wasted actions, and recognize dependencies between steps (for example, realizing that step C can't be done until step A and step B are both finished). That said, planning in agents isn't usually treated as a rigid, unchangeable script either — a good agent typically remains willing to revise its plan partway through if it perceives new information suggesting the original plan needs to change, which connects closely to the idea of reflection, which we'll get to shortly. The most effective agent designs tend to treat planning as a flexible, living guide, rather than a fixed set of instructions that must be followed no matter what.

07 Acting

Acting refers to the actual execution step in the agent loop — the point where the agent moves from thinking (reasoning and planning) to actually doing something in the real world (or at least, in whatever environment it's operating within).

This is the part where a tool actually gets called — a web search actually gets run, a piece of code actually gets executed, a file actually gets written, an email actually gets sent. Up until this point, everything the agent has done has essentially been internal thinking; acting is the step where that thinking translates into a real, concrete effect outside the agent itself.

There are a couple of important practical considerations around the acting step worth understanding. First, actions need to be **clearly defined** — each tool an agent can use typically has a specific, well-documented format for exactly what information it needs and what it will do with it, so the agent (really, the underlying language model) can reliably use it correctly, rather than the agent trying to take some vague, poorly specified action that the surrounding system doesn't know how to actually carry out. Second, some actions are **reversible and low-risk** (like performing a search, which has essentially no real-world consequence beyond retrieving some information), while others are **irreversible or higher-risk** (like sending an email, deleting a file, or making a purchase) — and well-designed agent systems often build in extra safeguards, like requiring explicit human confirmation, specifically around these higher-risk categories of actions, precisely because an agent autonomously taking a wrong, irreversible action can cause real, tangible harm in a way that an agent generating a slightly wrong sentence of text generally cannot.

08 Reflection

Reflection refers to an agent's ability to look back at what it's done so far — its previous actions and their results — and evaluate whether things are actually going well, whether the original plan still makes sense, and whether any adjustments are needed.

This is a genuinely important capability, because without it, an agent can end up in unproductive situations — repeating the same failed action over and over, continuing to follow an original plan even after it's become clear that plan won't work, or declaring a task "complete" when it actually hasn't fully accomplished what was asked. Reflection is what allows an agent to catch and correct these kinds of problems.

In practice, reflection often involves the agent (again, typically through the underlying language model generating text that evaluates the situation) explicitly asking itself questions like: did that last action actually produce the result I expected, or did something go wrong? Am I making genuine progress toward the overall goal, or am I stuck? Does my original plan still make sense given everything I've learned so far, or does it need to change? Have I actually achieved the goal now, or is there still more to do?

Some more advanced agent designs build in dedicated, explicit reflection steps — deliberately pausing after a set of actions specifically to evaluate progress before continuing — rather than only reasoning about the immediate next step in a purely forward-moving way. This kind of deliberate self-checking tends to make agents noticeably more reliable on longer, more complex tasks, since it gives the agent a real opportunity to catch and correct its own mistakes along the way, rather than blindly plowing forward on a flawed approach until it runs out of steps or time.

09 Memory

Memory refers to an agent's ability to retain and make use of information over time — both within a single task, and sometimes across entirely separate tasks or conversations that happen at different points in time.

It's useful to think about agent memory in two broad categories. **Short-term memory** (sometimes called working memory) refers to information the agent needs to keep track of just within the current task or conversation — what the original goal was, what actions it's already taken, what results those actions produced, and generally the whole running history of the current session. Without this, an agent would essentially forget what it was doing partway through a multi-step task, repeating actions it had already completed or losing track of its own original goal.

Long-term memory refers to information an agent retains and can draw on across completely separate sessions, potentially persisting for a very long time. This might include things like a user's stated preferences from a previous conversation, general facts the agent has learned and deemed worth remembering, or a record of past tasks and their outcomes that might be useful context for future, related tasks. This connects directly back to the RAG and vector database concepts we covered earlier — long-term agent memory is very often implemented, under the hood, using exactly the same kind of vector database and semantic search techniques we discussed, letting an agent efficiently search through a large history of past information and retrieve just the specific pieces relevant to its current situation, rather than needing to carry every single thing it's ever learned directly in its immediate working context at all times.

Getting memory right is a genuinely difficult and important design challenge in agent systems. Too little memory, and an agent seems forgetful, repeats itself, or loses track of important context. Too much undifferentiated memory crammed into its immediate context, and you run into the same context window and

cost problems we discussed in the RAG section, plus the practical risk of important information getting buried and effectively lost among a lot of less relevant accumulated history.

10 Environment

The environment refers to the actual space or system that an agent operates within and takes actions upon — essentially, the "world" the agent exists in and can perceive and affect, as we touched on briefly in the architecture and perception sections above.

Different agents operate in very different kinds of environments, depending on what they're built to do. A coding agent's environment might be a code repository and a command-line terminal, where it can read files, write code, and run commands. A web-browsing agent's environment might be an actual web browser it can control, letting it click links, fill out forms, and read webpage content. A customer service agent's environment might be a company's internal support ticketing system and knowledge base. A research agent's environment might be the open internet, accessed through search and page-reading tools.

The choice and design of an agent's environment matters enormously for how well it can actually accomplish its intended tasks, because the environment defines both what the agent can perceive and what actions it's actually capable of taking. An agent can only be as effective as the environment allows — a coding agent that can't actually run and test the code it writes is working with a significant handicap compared to one that can, since it has no reliable way to verify its own work through action and observation rather than pure reasoning alone. Well-designed agent environments generally aim to give the agent enough access and capability to genuinely accomplish realistic tasks, while also being appropriately sandboxed or restricted where necessary, so that mistakes or unexpected agent behavior don't cause unintended damage beyond the intended scope of the task.

11 Agent Lifecycle

The agent lifecycle refers to the overall beginning-to-end journey of a single agent task, from the moment it's given a goal to the moment that task is considered finished — essentially, zooming out from the repeating agent loop we discussed earlier to look at the whole shape of a task from start to finish.

A typical lifecycle generally looks something like this. It starts with **initialization** — the agent receives its goal or instruction, along with whatever initial context, tools, and environment access it's been given for this particular task. This is followed by the **main working phase** — the agent loop we described earlier runs repeatedly, cycling through perceiving, reasoning, planning, acting, and reflecting, gradually making progress toward the goal, potentially over many, many individual steps for a complex task. Throughout this phase, the agent's memory (both short and sometimes long-term) accumulates and gets used to inform each subsequent step.

Eventually, the lifecycle reaches a **termination point** — the task ends, for one of a few possible reasons. Ideally, this happens because the agent has determined, through its own reflection, that the goal has genuinely been accomplished. But a task can also end because it hits some predefined limit (like a maximum number of steps or a time limit, put in place specifically to prevent an agent from looping indefinitely if something has gone wrong), because it encounters an error it can't recover from, or because a human intervenes and stops it, whether to correct its course or because it's completed enough of the task for the human's actual purposes.

Finally, there's often a **wrap-up phase** — summarizing what was done, presenting final results back to the user, and potentially saving relevant information to long-term memory for future reference. Understanding this full lifecycle matters for building reliable agent systems, because it highlights the importance of having sensible limits and clear completion criteria built in from the start, rather than only designing for the optimistic case where everything goes smoothly and the agent naturally and correctly recognizes exactly when it's done.

12 Single Agent Systems

A single agent system, as the name suggests, involves just one agent working on a task from start to finish, using its own reasoning, tools, and memory, without coordinating with any other separate agents. This is the simplest and most common way agent systems are built, and it's worth understanding well before moving on to the more complex multi-agent systems discussed next.

In a single agent system, one language model, equipped with a defined set of tools and following the agent loop we described (perceive, reason, plan, act, reflect), handles the entire task on its own. For many practical tasks, this is genuinely sufficient — a single, well-equipped agent with access to the right tools can handle a surprisingly wide range of tasks, from answering research questions using web search, to writing and debugging code, to managing a structured multi-step workflow, all without needing to involve any other separate agents.

The main appeal of single agent systems is their relative simplicity — there's one central "brain" making all the decisions, one continuous stream of reasoning and memory to keep track of, and a generally more straightforward, easier-to-understand and easier-to-debug flow of what's happening and why, compared to coordinating multiple separate agents together. For a great many real-world applications, a well-designed single agent, given good tools and a well-designed environment, is entirely sufficient, and adding the extra complexity of multiple coordinating agents (discussed next) isn't actually necessary or beneficial.

13 Multi-Agent Systems

Multi-agent systems involve multiple separate AI agents working together, each potentially with its own distinct role, area of focus, and set of tools, coordinating with each other to accomplish a larger overall goal that might be difficult for a single agent to handle well on its own.

The core idea draws an analogy to how humans often organize complex work — rather than expecting one single person to be a genuine expert at absolutely everything involved in a large project, we often instead assemble a team where each person specializes in a particular piece, and the team as a whole coordinates and communicates to accomplish something none of them could do as well entirely alone. Multi-agent systems apply this same basic idea to AI agents.

A common pattern is having a **coordinator or manager agent** that receives the overall goal, breaks it down into smaller sub-tasks, and delegates each sub-task out to other, more specialized agents — for example, a manager agent might delegate a research sub-task to a dedicated research agent, delegate a writing sub-task to a dedicated writing agent, and delegate a fact-checking sub-task to a dedicated review agent, and then combine and coordinate all of their individual outputs into one final, complete result. Other patterns involve agents working more as peers, communicating directly with each other, debating different approaches, or reviewing and critiquing one another's work before a final result is settled on.

The potential benefits of this approach include allowing each individual agent to be more narrowly focused and specialized (which can genuinely improve quality on that specific piece of work, similar to how a human specialist tends to be better at their specific area than a generalist trying to do everything), and the ability to potentially work on multiple different pieces of a larger task in parallel, at the same time, rather than everything having to happen strictly one step after another.

However, multi-agent systems also introduce real additional complexity and real costs worth being aware of. Coordination between agents can itself become a significant source of problems — agents can miscommunicate, misunderstand each other's outputs, or end up working at cross purposes toward misaligned sub-goals. Multi-agent systems are also generally more expensive to run (since you're often running several separate language model calls instead of just one) and can be considerably harder to debug and understand when something goes wrong, precisely because there are multiple separate decision-making components interacting with each other rather than one single, clear, traceable stream of reasoning. Because of this real added complexity and cost, multi-agent systems tend to make the most sense specifically for genuinely large, complex tasks that meaningfully benefit from real specialization or real parallel work — rather than being a default, go-to choice for every single agent-based project, especially in cases where a well-designed single agent, as discussed above, would actually handle the task perfectly well on its own.

Agent Frameworks

01 LangChain

LangChain is one of the earliest and most widely known frameworks in this whole space, and it's often the first name beginners encounter when they start learning about building with language models. At its core, LangChain provides a large collection of standardized building blocks — a consistent way to connect to many different language models, a consistent way to define and use tools, a consistent way to work with document loading and retrieval (which connects back to the RAG concepts we covered earlier), and pre-built patterns for common agent loops.

The real value LangChain offers is this standardization and breadth — instead of learning a completely different way to connect to OpenAI's models versus Anthropic's models versus Google's models, LangChain gives you one consistent interface that works across all of them, along with hundreds of ready-made integrations to different tools, databases, and services that other developers have already built and shared. This makes it a genuinely fast way to get an initial working prototype up and running, especially for beginners, because so much of the "glue code" connecting different pieces together has already been written for you.

The tradeoff, and something you'll hear experienced developers mention, is that LangChain's abstractions can feel like a lot of extra layers between you and what's actually happening underneath, which can make it harder to debug or customize once you need very precise control over your agent's exact behavior. Because of this, LangChain is often described today as the good starting point for connecting models and tools together, while more precise control over the actual agent's decision-making loop is increasingly handled by its sister framework, LangGraph, discussed next.

02 LangGraph

LangGraph is built by the same company as LangChain, and it exists specifically to solve a problem that LangChain's original, simpler approach struggled with: real-world agent behavior is often not a simple, straight line from start to finish — it involves loops, branches, and decisions that depend on what happened earlier.

LangGraph's core idea is to represent an agent's entire workflow as a graph — picture a flowchart made up of "nodes" (individual steps, like calling the language model, using a tool, or making a decision) connected by "edges" (the paths that connect one step to the next). Crucially, unlike a simple flowchart that only goes in one direction, LangGraph's graphs can loop back on themselves and branch conditionally based on what happened at an earlier step — meaning an agent can genuinely revisit a decision, try something again, or take a completely different path depending on the results it's seeing, which mirrors how real, complex agent behavior actually needs to work.

LangGraph is considered a "low-level" framework, meaning it gives you fine, precise control over exactly how your agent's loop behaves, rather than hiding a lot of that logic behind convenient but less flexible shortcuts. It's particularly well regarded for building agents that need to run for a long time, maintain their state reliably even if something restarts partway through, pause and wait for a human to review or approve a step before continuing (called "human-in-the-loop"), and coordinate multiple different agents working

together. Because of this focus on precise control and production reliability, LangGraph is widely used by companies building serious, production-grade agent systems, though it does come with a steeper learning curve for beginners compared to some of the simpler frameworks we'll cover.

03 AutoGen

AutoGen is a framework developed by Microsoft, and its defining focus, right from the start, has been multi-agent conversations — the idea of multiple separate AI agents actually talking to each other, back and forth, to solve a problem together, rather than a single agent working alone.

The central concept in AutoGen is representing each agent as a kind of "conversational" participant, similar to how you might picture participants in a group chat. You might set up one agent to act as an "assistant" that writes code, another agent to act as a "critic" that reviews and challenges that code, and another that acts as a "user proxy" representing the human's interests. These agents then have an actual back-and-forth conversation with each other — proposing solutions, critiquing them, revising them — before arriving at a final result, mimicking how a team of human collaborators might hash out a problem together through discussion rather than each person just working in total isolation.

This design makes AutoGen particularly well-suited to tasks that genuinely benefit from this kind of debate-and-refine dynamic — for example, code generation and review, where having a separate "critic" agent catch mistakes before finalizing an answer tends to produce more reliable results than a single agent just generating code once and stopping. AutoGen has become a popular choice specifically for research and production systems that are built fundamentally around this idea of multiple specialized agents collaborating through structured conversation, rather than being designed primarily around a single agent's own internal loop.

04 CrewAI

CrewAI takes the general idea of multi-agent collaboration and wraps it in a particularly approachable, role-based metaphor: instead of thinking about "nodes and edges" or abstract conversational participants, CrewAI has you think about building an actual "crew" of AI agents, each with a defined role, a specific goal, and a backstory, working together like a team of human employees with clearly assigned job responsibilities.

In CrewAI, you typically define each agent with a specific role (like "researcher" or "writer"), give it a clear goal (like "find the most accurate and current statistics on this topic"), and even a brief backstory that helps shape how it approaches its work (like "you are a meticulous, detail-oriented analyst who double-checks every fact"). You then define specific tasks that need to get done, and assign them to the appropriate agents in your crew, with the framework handling the coordination of getting each agent's output passed along to the next step in the process.

CrewAI has become popular particularly among developers who find this role-based, team-oriented way of thinking more intuitive and quicker to reason about than some of the lower-level graph-based or conversation-based approaches — it maps naturally onto how people already think about organizing a team to accomplish a project. It's often reached for by developers who want to get a working multi-agent system up and running relatively quickly, with less of the low-level configuration that a framework like LangGraph requires, in exchange for somewhat less fine-grained control over the exact underlying mechanics.

05 OpenAI Agents SDK

The OpenAI Agents SDK is OpenAI's own official framework, built specifically to work smoothly with their own models and their own approach to tool-calling and agent behavior. Since it comes directly from the same company that built the underlying language models, it tends to have particularly tight, well-supported integration with OpenAI-specific features, like their built-in tools for web search, code execution, and file handling.

The design philosophy behind this SDK tends to lean toward being relatively lightweight and straightforward compared to some of the larger, more sprawling frameworks like LangChain — it focuses on providing clean, well-documented core building blocks (like defining agents, tools, and the handoffs between multiple agents) without piling on a huge number of extra abstractions or third-party integrations. It also includes built-in support for tracing and debugging an agent's behavior, which is valuable for actually understanding what your agent did and why during a run, rather than treating that as an afterthought.

Because it's built and maintained by OpenAI directly, this SDK is a particularly natural choice for developers who are already committed primarily to building on OpenAI's models and want the most direct, well-supported path to using OpenAI's specific agent-related features, though (like most modern frameworks) it can generally be adapted to work with other model providers too if needed.

06 Google ADK

Google ADK (Agent Development Kit) is Google's own official, open-source framework for building agents, and it's designed with a particular emphasis on making agent development feel more like traditional, structured software engineering, rather than a looser, more experimental process. It's notably the same underlying framework that powers some of Google's own production agent products, which speaks to it being built for genuinely serious, enterprise-scale use.

A distinctive feature of ADK is its clear support for different types of agents working together in defined structures — it explicitly distinguishes between "LLM agents" (the reasoning "brains" that make decisions), "workflow agents" (which follow more fixed, predictable sequences of steps), and custom agents you define yourself, and it gives you structured ways to combine these into hierarchies, where a top-level agent can coordinate and delegate to more specialized sub-agents beneath it. It also comes with a genuinely built-in evaluation framework, letting you systematically test how well your agent is performing (connecting back to the RAG evaluation ideas we discussed earlier, but applied to agent behavior specifically), and strong support for artifact handling (letting agents manage files, images, and other documents as part of their work).

While ADK is optimized for and works especially smoothly with Google's own Gemini models and Google Cloud infrastructure, it's designed to be model-agnostic and can work with other model providers too. It's a strong choice for teams already building within the Google Cloud ecosystem, or teams that specifically want a more structured, enterprise-oriented approach to organizing multi-agent hierarchies with built-in evaluation tooling.

07 Semantic Kernel

Semantic Kernel is Microsoft's other major entry in this space (distinct from AutoGen, which we covered above, and which is increasingly being positioned to work alongside Semantic Kernel rather than as a completely separate competing option). Its most distinctive characteristic is that it's built to work well across

multiple programming languages — notably C#, Python, and Java — which sets it apart from many other frameworks in this list that are built primarily around a single language, usually Python.

This multi-language support makes Semantic Kernel a particularly natural fit for organizations, especially larger enterprises, that already have significant existing software built in C# or Java and want to add AI agent capabilities into that existing codebase without having to introduce an entirely separate Python-based system alongside it. The core concepts in Semantic Kernel revolve around "plugins" (reusable pieces of functionality an agent can call, similar to tools in other frameworks), "planners" (components that help figure out the sequence of steps needed to accomplish a goal), and "connectors" (standardized ways to link up to different AI models and external services).

Semantic Kernel is generally positioned as an enterprise-friendly framework, with a strong emphasis on things like security, compliance, and integration with existing enterprise software systems, making it a natural choice for larger, more traditionally structured organizations building AI agents into established business software, particularly in Microsoft-centric technology environments.

08 **PydanticAI**

PydanticAI comes from the team behind Pydantic, a hugely popular Python library that's widely used for validating and structuring data (making sure that the data flowing through your program is actually in the shape and format you expect it to be, and catching errors early if it's not). PydanticAI brings that same philosophy directly into the world of building AI agents.

The core appeal of PydanticAI is type safety and structure — when you're building an agent, you often want to be very sure about exactly what shape of data is coming out of the language model (for example, making sure a response is genuinely a valid, well-structured piece of data with the specific fields you expect, rather than just loose, unstructured text that you have to hope is formatted correctly). PydanticAI leans heavily on Pydantic's existing, well-established data validation approach to make this reliable and predictable, catching structural problems clearly and early rather than allowing malformed data to silently cause confusing bugs deeper in your application.

Because it's built by the same team behind such a widely trusted and widely used existing Python library, PydanticAI tends to appeal particularly to developers who already value strong typing and predictable, well-validated data structures in their code, and who want that same level of rigor and reliability extended naturally into their agent-building work, rather than feeling like a bolted-on afterthought.

09 **SmolAgents**

SmolAgents comes from Hugging Face, a company well known and widely respected in the open-source AI community, and its whole design philosophy is captured well by its name — it deliberately aims to be small, simple, and minimal, in fairly direct contrast to some of the larger, more feature-heavy frameworks we've discussed.

One particularly distinctive idea in SmolAgents is its emphasis on what's sometimes called "code agents" — rather than having an agent work by selecting from a fixed, predefined list of specific tools one at a time, a code agent instead has the language model write and execute actual code (usually Python) to accomplish its goal. The reasoning behind this approach is that writing genuine code tends to be a more flexible and

naturally powerful way for an agent to solve problems, compared to being constrained to only ever calling from a fixed menu of specifically predefined tool functions — code can combine logic, loops, and calculations far more freely than a rigid, predefined tool-calling structure typically allows.

SmolAgents tends to appeal to developers who want to deeply understand what their agent framework is actually doing under the hood, without wading through a large, complex codebase full of many layers of abstraction, and who are drawn to the more flexible, code-first approach to giving an agent real capability, rather than agents restricted to a narrower, more rigidly predefined set of specific tool calls.

10 Agno

Agno (previously known as Phidata, before it was rebranded) is a Python framework whose central selling point is a strong focus on being lightweight, fast, and genuinely simple to work with — its own documentation and community discussions repeatedly emphasize that it deliberately avoids some of the more complex abstractions (like the graph-based structures used in LangGraph) in favor of writing agents using plain, straightforward Python.

Agno provides a broad set of built-in capabilities out of the box — support for a very large number of different model providers so you're not locked into just one company's models, built-in memory and knowledge management (connecting back to the RAG concepts we discussed earlier), native support for multiple types of content beyond just text (like images, audio, and video), and the ability to combine individual agents into coordinated "teams" that work together, as well as more deterministic, rule-based "workflows" for situations where you want more predictable, structured control rather than letting an agent decide everything dynamically on its own.

A big part of Agno's appeal, especially as it's grown in popularity, is its emphasis on genuine production-readiness — it includes tooling for actually deploying, monitoring, and scaling agents once you've moved past the prototyping phase, rather than being purely a toolkit for quick experiments. This combination of a simple, minimal core developer experience alongside serious production infrastructure has made it an increasingly popular choice among developers who found some of the older, more established frameworks to feel unnecessarily heavy or complicated for what they were actually trying to build.

11 DSPy

DSPy takes a genuinely different, more academically-rooted approach compared to everything else on this list, and it's worth understanding because the underlying idea is quite clever. Most agent frameworks assume you'll write out your prompts (the actual instructions and wording given to the language model) mostly by hand, through trial and error, gradually tweaking the wording until you get good results. DSPy instead treats prompt-writing more like a machine learning problem that can be automatically optimized, rather than something a human should have to painstakingly hand-tune through guesswork.

With DSPy, you define what you want your agent or pipeline to accomplish in a more structured, declarative way — specifying the inputs, the desired outputs, and some examples of good performance — and DSPy's underlying system then automatically experiments with and refines the actual prompt wording and structure being sent to the language model, searching for the version that produces the best measured results according to your specified criteria, rather than you manually guessing and rewriting prompts by hand over and over.

This makes DSPy a particularly interesting choice for situations where getting the best possible, most reliable performance out of a specific, well-defined task genuinely matters, and where you're willing to invest some extra upfront effort in defining clear examples and evaluation criteria in exchange for a more rigorous, systematic, and often more effective process than manual prompt tweaking. It tends to appeal more to developers and researchers with a background or interest in machine learning practices, since its whole approach is fundamentally about treating prompt design as something to be measured and optimized, rather than an appeal to a purely conversational, hand-crafted framework style.

12 Mastra

Mastra fills a distinctive niche in this list because of a fairly simple but important reason: it's built specifically for TypeScript and the broader JavaScript ecosystem, whereas most of the other frameworks we've covered (LangChain, CrewAI, Agno, DSPy, SmolAgents) are primarily Python-based. This matters a lot in practice, because a huge amount of real-world web application development happens in JavaScript and TypeScript, and teams building agents that need to plug directly into an existing web app, with real-time streaming responses and modern web deployment, often find a Python-based framework to be an awkward fit alongside their existing codebase.

Mastra provides the now-familiar set of core agent-building pieces — agents that can use tools and decide their own sequence of actions, workflows for more structured, multi-step processes, built-in memory for maintaining context across conversations, and support for retrieval-augmented generation. It also puts a genuine emphasis on things especially relevant to web developers specifically, like straightforward integration with popular frontend frameworks, real-time streaming of an agent's responses directly into a user interface, and deployment options tailored to common modern web hosting platforms.

Mastra has grown a meaningful, dedicated following particularly among web development teams who want to build AI agents using the same language and tooling ecosystem they already use for the rest of their application, rather than needing to introduce and maintain an entirely separate Python-based service just to add agent capabilities into an otherwise all-JavaScript product.

13 Atomic Agents

Atomic Agents is a framework built around a particular design philosophy captured well by its name: breaking agent systems down into small, clearly defined, independent "atomic" pieces, rather than relying on large, more monolithic or heavily abstracted structures. The core idea is that each individual piece of an agent's functionality should be small, focused, well-defined, and easy to understand and test entirely on its own, similar in spirit to good general software engineering practice around writing small, focused, single-purpose functions rather than large, tangled, do-everything ones.

This approach tends to emphasize clear, explicit, strongly-typed interfaces between different components of an agent system, so that data flowing between one piece and the next is well-defined and predictable, reducing the kind of confusing, hard-to-debug behavior that can creep into agent systems where the flow of information between different steps is loosely defined or unclear. Rather than encouraging you to reach for large, pre-built, catch-all abstractions, Atomic Agents encourages composing your agent's behavior out of these smaller, individually well-understood, interchangeable building blocks.

This kind of framework tends to appeal particularly to developers who've had frustrating experiences with larger, more heavily-abstracted frameworks becoming difficult to debug or customize once a project grows more complex, and who specifically value a smaller, more transparent, more explicitly structured foundation

to build on top of, even if it means writing somewhat more of the surrounding logic themselves compared to a larger framework that tries to handle more of it automatically on your behalf.

Agent Patterns

01 ReAct Pattern

ReAct is one of the most foundational and widely used patterns in the entire agent world, and its name is a combination of two words: "Reasoning" and "Acting." The core idea is refreshingly simple once you see it: instead of an agent either purely thinking through a problem in its head with no real-world actions, or purely taking actions with no real thinking behind them, ReAct has the agent alternate between the two, over and over, in a tight back-and-forth loop.

Here's what this actually looks like in practice. The agent first writes out a short piece of reasoning — essentially thinking out loud about what it currently understands and what it should do next (something like "I need to find out the current weather in Paris before I can answer this question"). Then, based on that reasoning, it takes a single action (like calling a weather-lookup tool). It then observes the result of that action (the actual weather data that comes back), and this observation feeds directly into its next round of reasoning ("Okay, it's currently raining in Paris, so now I should check..."), and the cycle repeats.

Why does this alternating structure work so well? Because it closely mirrors how a thoughtful human actually works through an unfamiliar problem — you don't typically plan out every single step perfectly in advance with zero regard for what you'll actually discover along the way; instead, you think a little, act a little, see what happens, and adjust your thinking based on what you just learned. This makes ReAct agents noticeably more reliable and less prone to going down completely wrong paths compared to earlier, simpler designs where an agent either tried to plan everything in one shot upfront, or took actions without any explicit reasoning behind them at all. Because of how well this pattern works and how simple it is to understand, ReAct became something of a foundational default — many other, more advanced patterns are really just variations or extensions built on top of this same core reason-then-act loop.

02 Planner Pattern

The Planner pattern addresses a specific limitation that can show up in a purely step-by-step approach like ReAct — namely, that only ever thinking about the single very next step, without any broader view of the whole task, can sometimes lead an agent to work inefficiently, backtrack unnecessarily, or miss dependencies between different parts of a larger task.

In the Planner pattern, before diving into execution, the agent (or a dedicated planning component) first produces a fuller plan — an explicit, ordered breakdown of the major steps it believes are needed to accomplish the overall goal, thought through upfront, before any of those steps are actually carried out. Picture the difference between wandering into a grocery store with no list and just grabbing whatever catches your eye as you walk down each aisle, versus writing out your shopping list at home first, organized by section, so you know exactly what you need and can move through the store efficiently without doubling back for something you forgot.

This upfront planning step tends to make an agent noticeably more effective at complex, multi-part tasks, because it surfaces the overall shape and dependencies of the problem early on — for example, recognizing that a particular step can't be done until an earlier one is finished, or noticing that two parts of the task can actually be tackled independently and don't need to happen in a strict sequence. A well-designed Planner

pattern typically isn't treated as a rigid, unchangeable script either — the plan is usually revisited and adjusted as the agent learns new things during execution, but having that initial roadmap still provides valuable structure and direction compared to figuring out each step completely from scratch as you go.

03 Executor Pattern

The Executor pattern is closely related to, and often directly paired with, the Planner pattern above — in fact, the two are frequently used together as a matched set, which is why it's worth understanding them side by side. Where the Planner's job is to figure out *what* needs to be done and in what order, the Executor's job is to actually *carry out* each of those individual steps.

This separation of responsibilities is a genuinely useful design choice, and it mirrors a very common pattern in how human teams organize work too — think of a project manager who lays out the overall plan and sequence of tasks, versus the specific team members who actually go and do the hands-on work of each individual task. In an agent system built this way, a Planner component produces the ordered list of steps, and then a separate Executor component (which might even be a completely different, more specialized agent, or several different specialized agents handling different types of steps) takes each individual step from that plan and actually performs it — calling the necessary tools, gathering the necessary information, and producing the actual result for that specific piece of the work.

The benefit of clearly separating planning from execution like this is that each part can be optimized and reasoned about somewhat independently — the Planner can focus purely on the higher-level, strategic question of "what needs to happen and in what order," without getting bogged down in the messy, detailed specifics of exactly how each individual step gets carried out, while the Executor can focus entirely on reliably carrying out one specific, well-defined step at a time, without needing to hold the entire broader plan in its head simultaneously. This division of labor tends to make the overall system easier to build, easier to debug (since you can look separately at whether the plan itself was flawed, versus whether a specific step was executed poorly), and easier to extend over time.

04 Router Pattern

The Router pattern addresses a common situation: not every incoming request or task should necessarily be handled the exact same way, and sending every single request down one single, identical processing path is often wasteful or simply the wrong approach for certain types of requests.

The core idea is straightforward — a "router" component sits at the front of the system, looks at an incoming request, and decides which specific path, tool, or specialized agent is actually the right one to handle it, before actually sending it there. Think of it like a receptionist at a large office building who greets everyone coming in and directs them to the correct department based on what they actually need, rather than everyone being funneled through the exact same single desk regardless of their purpose for visiting.

A concrete example makes this clearer: imagine a customer support agent system that needs to handle several very different kinds of incoming questions — a simple question about store hours, a complex technical troubleshooting issue, and a billing dispute. A Router pattern would first look at each incoming question and decide which of these categories it falls into, and then send it to the specifically appropriate path — perhaps a very lightweight, fast response for the simple store-hours question (since it doesn't need heavy reasoning), a more capable and specialized troubleshooting agent for the technical issue, and a completely different specialized agent (perhaps one with access to different, more sensitive tools) for the billing dispute. This pattern is valuable both for keeping costs and speed reasonable (not every request

needs the most powerful, most expensive processing path) and for genuinely improving quality (a specialized path built specifically for billing disputes will likely handle that type of request better than one generic, catch-all approach trying to handle everything equally well).

05 Supervisor Pattern

The Supervisor pattern is a specific way of organizing multiple agents working together, and it maps closely onto a familiar, intuitive structure: a manager overseeing a team of workers, where the manager doesn't necessarily do the hands-on work themselves, but is responsible for coordinating, delegating, and overseeing the work of others.

In this pattern, there's one central "supervisor" agent that receives the overall goal and is responsible for breaking it down and delegating specific pieces of work out to other, more specialized "worker" agents beneath it — similar in spirit to the manager-and-specialists metaphor we saw with CrewAI in the previous explanation, though this is the general underlying pattern rather than any one specific framework's particular implementation of it. The supervisor typically monitors the progress and results coming back from each worker agent, decides what to do with those results (perhaps passing one worker's output along as input to another worker, or deciding a particular piece of work needs to be redone), and ultimately assembles everything into a final, complete result once all the necessary pieces have come together.

This pattern tends to work especially well for tasks that naturally break down into clearly distinct areas of specialization, where having one coordinating "brain" tracking the overall goal and orchestrating the different specialized pieces produces better, more coherent results than either a single generalist agent trying to do everything itself, or a looser group of agents with no clear coordinating authority at all. The tradeoff is that the supervisor itself becomes an important, central piece that the whole system depends on — if the supervisor makes a poor delegation decision or fails to properly integrate the results coming back from its workers, the quality of the entire system's output suffers, even if each individual specialized worker agent performed its own specific piece of the task perfectly well.

06 Reflection Pattern

We touched on the general concept of reflection back in the AI Agent Fundamentals explanation, but it's worth covering it here specifically as a named, reusable pattern, because of just how broadly useful and widely applied it's become across many different types of agent systems.

The Reflection pattern involves explicitly building a dedicated self-review step into an agent's process — after producing some piece of work (an answer, a piece of code, a plan, a draft of some content), the agent (or sometimes a separate, distinct "critic" component) deliberately pauses to evaluate that work against the original goal, actively looking for mistakes, gaps, or ways it could be improved, before treating the work as genuinely finished.

A simple, relatable comparison: think about the difference between dashing off a quick email and hitting send immediately, versus writing a first draft, then deliberately rereading it once before sending, specifically looking for typos, unclear phrasing, or a tone that doesn't quite land the way you intended. The Reflection pattern builds this same kind of deliberate, structured second look directly into an agent's process, rather than treating whatever the agent produces on its very first attempt as automatically good enough. In practice, this pattern often noticeably improves the quality of an agent's final output, particularly on more

complex, nuanced tasks like writing or coding, precisely because it gives the system an explicit, structured opportunity to catch and correct its own mistakes, rather than relying entirely on getting everything exactly right on the very first attempt with no chance for self-correction at all.

07 Debate Pattern

The Debate pattern takes the general idea behind reflection and extends it into a genuinely multi-agent setting — instead of one single agent reviewing and critiquing its own work, you set up two or more separate agents to actively argue different positions or perspectives on a problem, with the goal of arriving at a better final answer through this structured back-and-forth exchange than any single agent would likely reach entirely on its own.

Here's the general idea in practice: you might set up one agent to argue in favor of a particular answer or approach, and a second, separate agent specifically tasked with challenging that position, poking holes in the reasoning, or proposing genuine alternatives. These two agents go back and forth over several rounds, each responding to and pushing back against the other's points, and either a set number of rounds are run before a final answer is settled on, or a separate "judge" component (which might be a third agent, or a human) reviews the full exchange and decides which position, or what combination of positions, actually represents the strongest, best-supported final answer.

The underlying reasoning behind this pattern is that a single agent working alone can sometimes settle too quickly and too confidently on an answer that seems reasonable on the surface but actually has real flaws or blind spots it never considered, simply because nothing forced it to seriously entertain a genuine counter-argument. Deliberately building in this structured, adversarial back-and-forth tends to surface weaknesses in an initial answer that a single agent working in isolation would likely have missed entirely, similar to how a well-run human debate, or a genuinely rigorous peer review process, often produces a stronger, more carefully vetted final conclusion than one person simply thinking through a problem entirely by themselves with no pushback at all.

08 Human-in-the-Loop

Human-in-the-loop refers to a pattern where a real human is deliberately built into an agent's process at one or more specific points, rather than the agent running from start to finish entirely on its own with absolutely no human involvement or oversight along the way.

This matters more than it might initially seem, because fully autonomous agents — ones that make every single decision and take every single action entirely on their own — carry real risk, especially for actions that are irreversible, high-stakes, or genuinely hard to fully specify correct behavior for in advance (things we touched on briefly back in the "Acting" section of the agent fundamentals explanation). Human-in-the-loop patterns address this by deliberately building in specific checkpoints where the agent pauses and waits for explicit human review or approval before proceeding — for example, an agent might be allowed to freely search the web and draft an email entirely on its own, but be required to pause and get explicit human confirmation before that email is actually sent, since sending it is a real-world, hard-to-undo action with genuine consequences.

There are a few different common ways this gets implemented in practice. Some systems require **approval before specific high-risk actions** (like the email-sending example above). Others build in periodic **check-ins during a long, multi-step task**, giving a human the chance to review progress so far and redirect the agent's approach if it's heading somewhere unintended, rather than only discovering a problem after the entire task

has already finished. And some systems specifically use humans to **provide missing information or make a judgment call the agent genuinely isn't well-equipped to make on its own** — for example, an agent handling a customer complaint might be able to gather all the relevant facts and draft a proposed resolution on its own, but still hand the final decision on whether to actually offer a refund over to an authorized human, particularly for judgment calls that carry real financial or relationship consequences. This pattern represents a deliberate, thoughtful tradeoff — giving up some amount of full autonomy and speed, in exchange for meaningfully greater safety, oversight, and trustworthiness, particularly for the kinds of tasks where a mistake would be genuinely costly or difficult to walk back.

09 Swarm Agents

The Swarm pattern describes a genuinely different way of organizing multiple agents, compared to the more centrally coordinated approaches we discussed earlier, like the Supervisor pattern. Rather than having one central, coordinating "brain" directing and overseeing everything, a swarm consists of many individual agents operating with a much more decentralized structure, where overall useful behavior emerges from the interaction of many simpler, more independent agents, rather than being explicitly directed from the top down by one central authority.

The name deliberately draws inspiration from swarm behavior seen in nature — think of how a flock of birds or a colony of ants can accomplish surprisingly sophisticated, coordinated collective behavior, even though no single bird or ant is actually directing the whole group's overall strategy; each individual is simply following relatively simple local rules and reacting to its immediate neighbors, and the sophisticated group-level behavior emerges naturally out of these many simple, local, decentralized interactions.

In an AI agent context, a swarm-based system typically involves many individual agents that can hand tasks off to each other somewhat fluidly and dynamically, without necessarily requiring approval or coordination from one fixed central supervisor for every single decision or handoff. This kind of structure can make a system more flexible and resilient in certain situations — since there's no single central bottleneck or point of failure that every decision has to pass through — but it also tends to be considerably harder to predict and control precisely, and correspondingly harder to debug when something goes wrong, since there isn't one clear, central place to look for exactly why the overall system's collective behavior ended up the way it did. Because of this tradeoff, swarm-style patterns tend to be used more selectively, for specific situations where flexibility and resilience are especially valuable, rather than being the default, go-to structure for most everyday multi-agent applications.

10 Hierarchical Agents

The Hierarchical pattern organizes multiple agents into distinct, structured layers or levels, somewhat like an organizational chart in a traditional company, with higher-level agents overseeing broader, more strategic goals, and lower-level agents handling narrower, more specific pieces of work, often several layers deep.

This extends the basic idea behind the Supervisor pattern we discussed earlier, but goes further by allowing multiple layers of this same structure to stack on top of each other. For example, picture a top-level agent responsible for an overall broad goal, which delegates out to several mid-level supervisor agents, each responsible for a distinct major sub-area of that goal, and each of those mid-level agents, in turn, delegates further down to their own specific worker agents handling narrow, specific tasks within their particular sub-area — much like how a large company might have a CEO overseeing several department heads, each of whom oversees their own team of individual specialists actually doing the hands-on work.

This layered structure tends to become especially valuable for genuinely large, complex tasks or systems, where trying to have a single flat structure (one supervisor directly managing dozens of individual worker agents all at once, with no intermediate layers) would become unwieldy and difficult for that one supervisor to meaningfully track and coordinate all at once. By breaking things into layers, each individual agent at any given level only needs to reason about and manage a manageable, appropriately-scoped piece of the overall problem — a mid-level supervisor only needs to track its own specific sub-area and the handful of workers reporting to it, rather than needing full visibility and direct control over the entire system's complete complexity all at once. The tradeoff, similar to real organizational hierarchies, is that information and decisions have to pass through more intermediate layers to move between the very top and the very bottom of the structure, which can introduce more overall latency and more opportunities for something to get lost, distorted, or delayed somewhere along that longer chain, compared to a flatter, more directly coordinated structure.

11 Event-Driven Agents

The Event-Driven pattern represents a different way of thinking about when and why an agent actually takes action, compared to the patterns we've discussed so far. Rather than an agent running in response to a direct, deliberate request (like a user typing in a question and waiting for a response), an event-driven agent instead sits and waits, continuously monitoring for specific events happening in its environment, and automatically springs into action whenever a relevant event actually occurs.

Think of the difference between a customer support agent that only ever responds when a customer directly types in a question, versus a monitoring agent that's constantly watching a company's systems in the background, and automatically kicks off an investigation the moment it detects something like a server error rate spiking, or a payment failing, without anyone having to explicitly ask it to check at that particular moment. The "events" that trigger this kind of agent can come from all sorts of sources — a new email arriving, a file being uploaded to a shared folder, a specific value in a database changing, a scheduled time being reached, or a notification coming in from some other separate software system entirely.

This pattern is particularly well suited to tasks that fundamentally involve ongoing monitoring, or reacting promptly to things that happen unpredictably, rather than tasks that are naturally initiated by a direct, deliberate human request each time. It does introduce its own particular design considerations worth being aware of — an event-driven agent needs a genuinely reliable way to actually detect and correctly interpret the relevant events in the first place, and careful thought needs to go into avoiding situations where the agent ends up firing off actions too frequently or on genuinely irrelevant or unimportant events, which could otherwise create unnecessary noise, unnecessary cost, or even actively unhelpful, unwanted behavior if the triggering and filtering logic isn't well designed and properly tuned to the specific situation.